2.9. Автоматизация обработки данных

2. Файлы, данные, простая обработка и визуализация

В этом уроке

- Обрабатываем данные и создаем отчет
- Автоматизируем процесс

Практическое задание

Разобрать скрипт generate_report.py

Автоматизировать цикл:

- загрузка → фильтрация → агрегирование → отчёт

Работать с:

- datetime — фильтрация по датам
- os — работа с папками и файлами
- csv — работа с данными вручную
- pandas — быстрая обработка и агрегация

Структура проекта

```
project/
├── data/
│   ├── vulnerabilities_2025-06-12.csv
│   └── vulnerabilities_2025-06-13.csv
├── reports/
│   └── ...
├── generate_report.py
└── report_template.txt
```

timestamp	severity	system	message
2025-06-13 10:00:01	CRITICAL	server-01	Remote code execution
2025-06-13 10:05:23	HIGH	server-02	SQL injection
2025-06-13 11:15:45	MEDIUM	server-01	Outdated library

Шаблон отчёта (report_template.txt)

```
Отчёт по уязвимостям за {date}

Системы:
{systems}

Статистика по уровням:
- CRITICAL: {critical}
- HIGH:     {high}
- MEDIUM:   {medium}
- LOW:      {low}

Общее количество: {total}
```

Практическое задание

Загрузка и объединение CSV-файлов (pandas)

```
import pandas as pd
import glob

all_files = glob.glob("data/*.csv")
df_list = [pd.read_csv(f) for f in all_files]
data = pd.concat(df_list)
```

Автоматическая загрузка всех файлов из папки data/
Используем concat для объединения всех таблиц

Практическое задание

Фильтрация по дате с datetime

```
from datetime import datetime

data['timestamp'] = pd.to_datetime(data['timestamp'])
target_day = '2025-06-13'
filtered = data[data['timestamp'].dt.date == datetime.strptime(target_day, "%Y-%m-%d").date()]
```

Преобразуем строку в datetime
Отфильтровываем строки по дате

Практическое задание

Агрегация по уровню угроз

```
counts = filtered['severity'].value_counts().to_dict()
total = len(filtered)
systems = set(filtered['system'])
```

Считаем количество по каждой категории (CRITICAL, HIGH и т.д.)
Выделяем все уникальные системы

Практическое задание

Формирование отчета из шаблона

```
with open("report_template.txt", "r", encoding="utf-8") as f:
    template = f.read()

report = template.format(
    date=target_day,
    systems="\n".join(systems),
    critical=counts.get("CRITICAL", 0),
    high=counts.get("HIGH", 0),
    medium=counts.get("MEDIUM", 0),
    low=counts.get("LOW", 0),
    total=total
)
```

Практическое задание

Сохранение отчёта

```
import os

os.makedirs("reports", exist_ok=True)
with open(f"reports/report_{target_day}.txt", "w", encoding="utf-8") as f:
    f.write(report)
```

Скринкаст

Итоги урока

- Автоматизировали создание отчетов